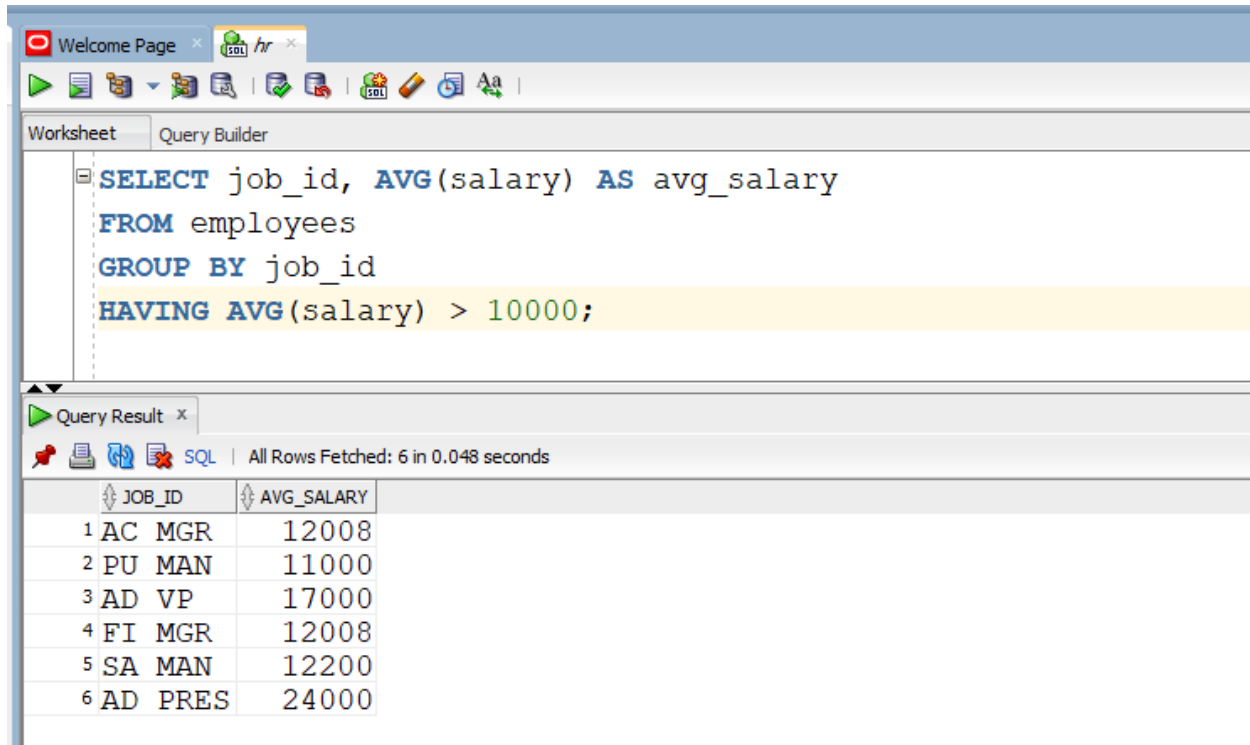


DBS LAB 6

IN LAB TASK

TASK 1:



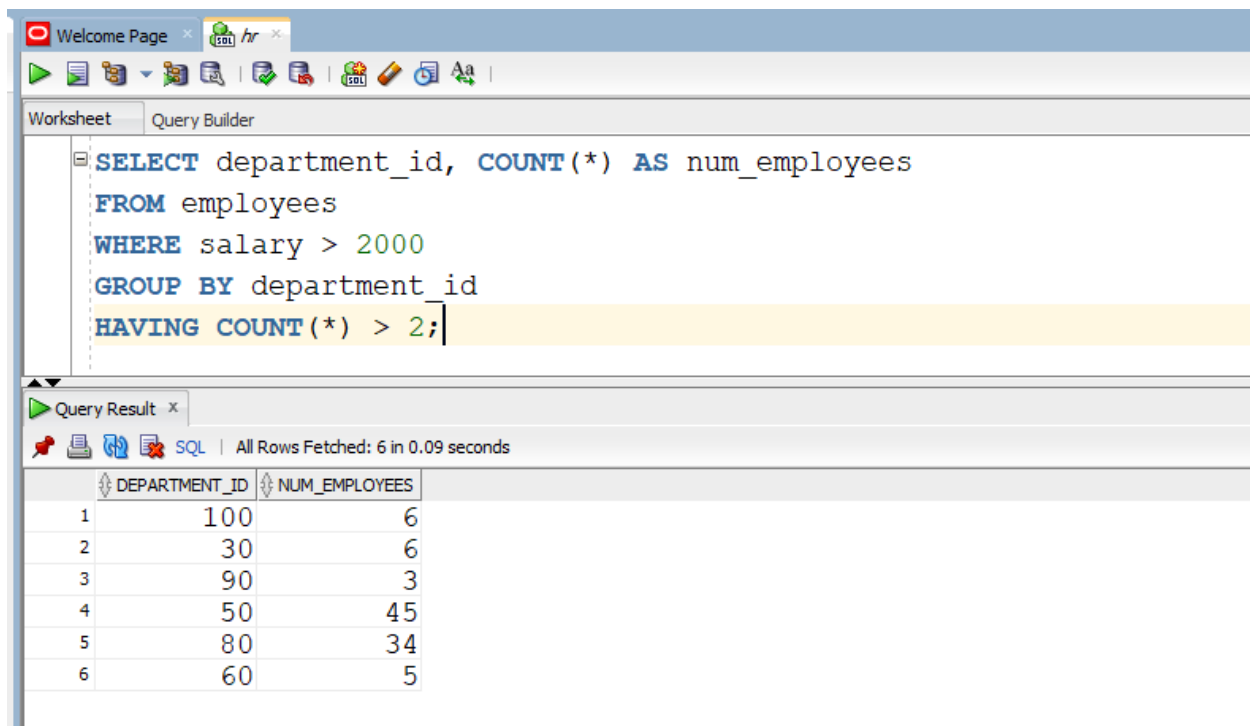
The screenshot shows a SQL IDE interface. The top toolbar includes icons for running queries, saving, and other standard functions. The 'Query Builder' tab is active, displaying the following SQL query:

```
SELECT job_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY job_id
HAVING AVG(salary) > 10000;
```

Below the query editor, the 'Query Result' tab shows the execution results. It indicates that all rows were fetched in 0.048 seconds. The results are displayed in a table with two columns: JOB_ID and AVG_SALARY.

	JOB_ID	AVG_SALARY
1	AC MGR	12008
2	PU MAN	11000
3	AD VP	17000
4	FI MGR	12008
5	SA MAN	12200
6	AD PRES	24000

TASK 2:



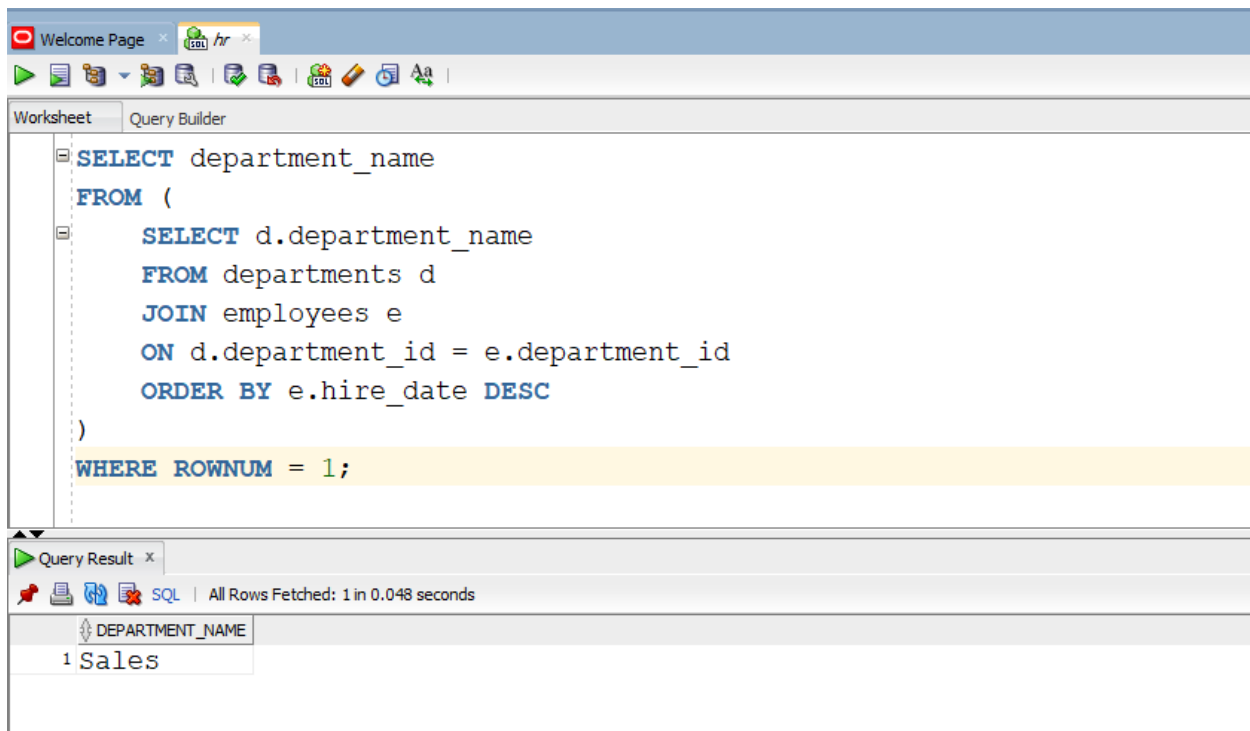
The screenshot shows the SQL Developer interface with a query in the Query Builder. The query is:

```
SELECT department_id, COUNT(*) AS num_employees
FROM employees
WHERE salary > 2000
GROUP BY department_id
HAVING COUNT(*) > 2;
```

The Query Result pane shows the following data:

	DEPARTMENT_ID	NUM_EMPLOYEES
1	100	6
2	30	6
3	90	3
4	50	45
5	80	34
6	60	5

TASK 3:



The screenshot shows the SQL Developer interface with a query in the Query Builder. The query is:

```
SELECT department_name
FROM (
  SELECT d.department_name
  FROM departments d
  JOIN employees e
  ON d.department_id = e.department_id
  ORDER BY e.hire_date DESC
)
WHERE ROWNUM = 1;
```

The Query Result pane shows the following data:

	DEPARTMENT_NAME
1	Sales

TASK 4:

Welcome Page

Worksheet Query Builder

```
CREATE INDEX emp_index
ON employees(employee_id);

CREATE INDEX job_hist_index
ON job_history(employee_id);

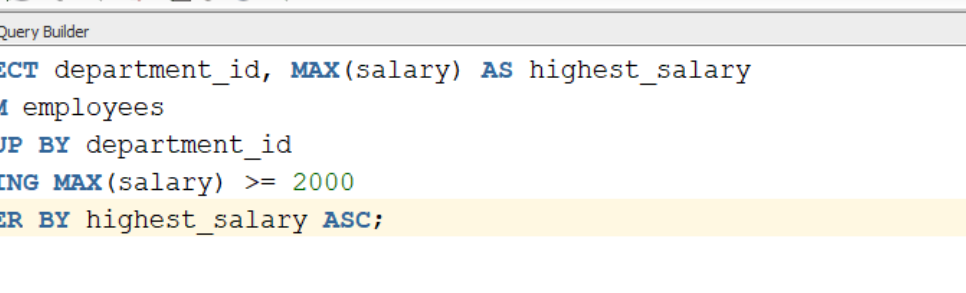
SELECT *
FROM employees
WHERE employee_id NOT IN (
    SELECT employee_id FROM job_history
);
```

Query Result x

SQL | Fetched 50 rows in 0.089 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17-JUN-03	AD PRES	24000	(null)	(null)	90
2	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-06	IT PROG	9000	(null)	102	60
3	104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-07	IT PROG	6000	(null)	103	60
4	105	David	Austin	DAUSTIN	590.423.4569	25-JUN-05	IT PROG	4800	(null)	103	60
5	106	Valli	Pataballa	VPATABAL	590.423.4560	05-FEB-06	IT PROG	4800	(null)	103	60
6	107	Diana	Lorentz	DLORENTZ	590.423.5567	07-FEB-07	IT PROG	4200	(null)	103	60
7	108	Nancy	Greenberg	NGREENBE	515.124.4569	17-AUG-02	FI MGR	12008	(null)	101	100
8	109	Daniel	Faviet	DFAVIET	515.124.4169	16-AUG-02	FI ACCOUNT	9000	(null)	108	100
9	110	John	Chen	JCHEN	515.124.4269	28-SEP-05	FI ACCOUNT	8200	(null)	108	100
10	111	Ismael	Sciarra	ISCIARRA	515.124.4369	30-SEP-05	FI ACCOUNT	7700	(null)	108	100
11	112	Jose Manuel	Urman	JMURMAN	515.124.4469	07-MAR-06	FI ACCOUNT	7800	(null)	108	100
12	113	Luis	Popp	LPOPP	515.124.4567	07-DEC-07	FI ACCOUNT	6900	(null)	108	100
13	115	Alexander	Khoo	AKHOO	515.127.4562	18-MAY-03	PU CLERK	3100	(null)	114	30
14	116	Shelli	Baida	SBAIDA	515.127.4563	24-DEC-05	PU CLERK	2900	(null)	114	30
15	117	Sigal	Tobias	STOBIAS	515.127.4564	24-JUL-05	PU CLERK	2800	(null)	114	30
16	118	Guy	Himuro	GHIMURO	515.127.4565	15-NOV-06	PU CLERK	2600	(null)	114	30
17	119	Karen	Colmenares	KCOLMENA	515.127.4566	10-AUG-07	PU CLERK	2500	(null)	114	30
18	120	Matthew	Weiss	MWEISS	650.123.1234	18-JUL-04	ST MAN	8000	(null)	100	50
19	121	Adam	Finn	AFFINN	650.123.2234	10-APR-05	ST MAN	8200	(null)	100	50

TASK 5:



The screenshot shows a database query editor with a 'Query Builder' tab. The query is as follows:

```
SELECT department_id, MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id
HAVING MAX(salary) >= 2000
ORDER BY highest_salary ASC;
```

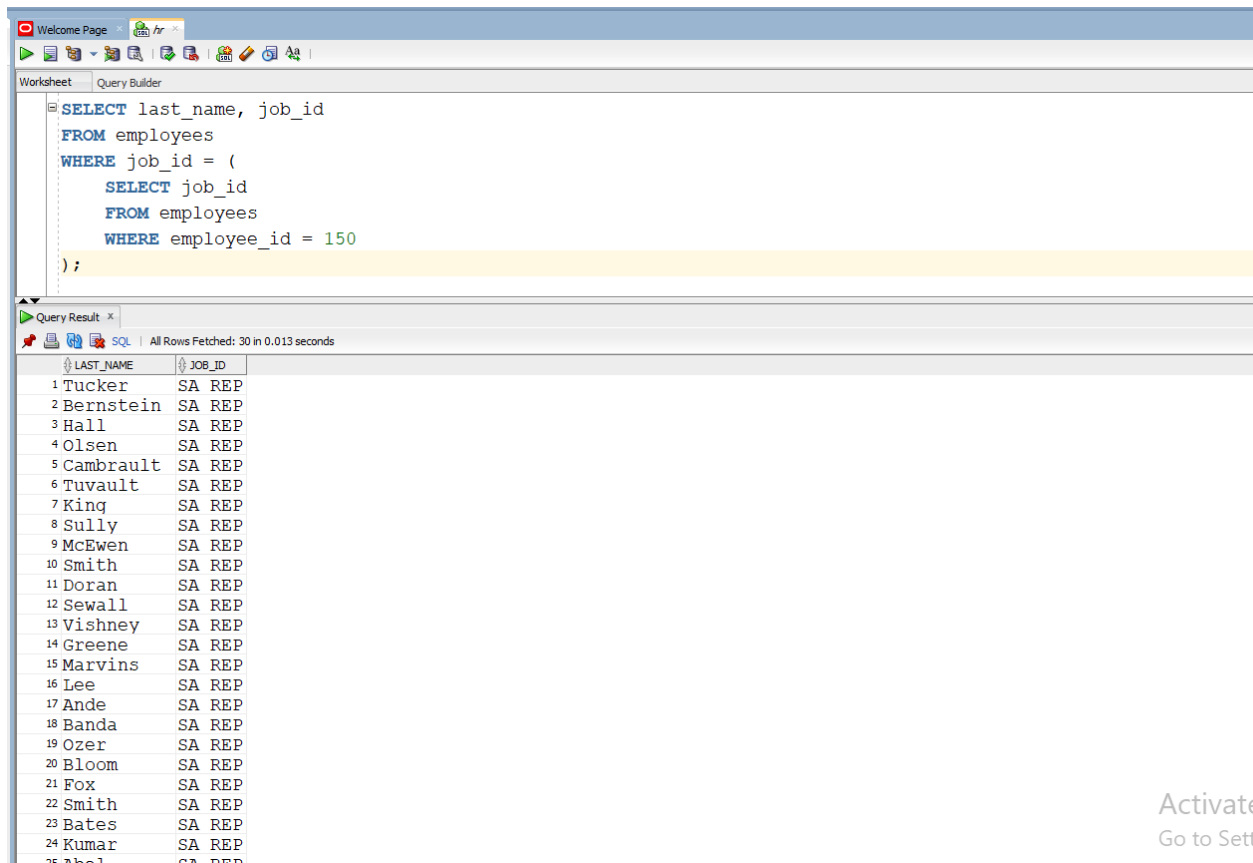
Below the query editor, the 'Query Result' tab is active, displaying the results of the query. The results are shown in a table with two columns: 'DEPARTMENT_ID' and 'HIGHEST_SALARY'. The table contains 11 rows of data, sorted by 'HIGHEST_SALARY' in ascending order.

	DEPARTMENT_ID	HIGHEST_SALARY
1	10	4400
2	40	6500
3	(null)	7000
4	50	8200
5	60	9000
6	70	10000
7	30	11000
8	100	12008
9	110	12008
10	80	14000
11	90	24000

TASK 6:

[illegible]

TASK 7:



The screenshot shows a SQL query editor window with a query that selects the last name and job ID of employees whose job ID matches the job ID of employee 150. The query is as follows:

```
SELECT last_name, job_id
FROM employees
WHERE job_id = (
    SELECT job_id
    FROM employees
    WHERE employee_id = 150
);
```

Below the query editor, the 'Query Result' window displays the results of the query. It shows 25 rows of data, each with a row number, last name, and job ID. The job ID for all rows is 'SA REP'.

	LAST_NAME	JOB_ID
1	Tucker	SA REP
2	Bernstein	SA REP
3	Hall	SA REP
4	Olsen	SA REP
5	Cambrault	SA REP
6	Tuvault	SA REP
7	King	SA REP
8	Sully	SA REP
9	McEwen	SA REP
10	Smith	SA REP
11	Doran	SA REP
12	Sewall	SA REP
13	Vishney	SA REP
14	Greene	SA REP
15	Marvins	SA REP
16	Lee	SA REP
17	Ande	SA REP
18	Banda	SA REP
19	Ozer	SA REP
20	Bloom	SA REP
21	Fox	SA REP
22	Smith	SA REP
23	Bates	SA REP
24	Kumar	SA REP
25	Abel	SA REP

On the right side of the screen, there is a button labeled 'Activate Go to Sett'.

TASK 8:

The screenshot shows the Oracle SQL Developer interface. The top pane displays a SQL script with two commands: `CREATE TABLE Job_History1` and `AS SELECT * FROM job_history WHERE 1=2;`. The bottom pane shows the execution results, including error messages for both commands.

Query Builder

```
CREATE TABLE Job_History1
AS SELECT * FROM job_history WHERE 1=2;
```

Query Result | **Script Output**

Task completed in 0.26 seconds

Error at Command Line : 4 Column : 13

Error report -

SQL Error: ORA-00942: table or view does not exist

00942. 00000 - "table or view does not exist"

*Cause:

*Action:

Error starting at line : 4 in command -

```
INSERT INTO Job_History1
SELECT *
FROM hr.job_history
WHERE end_date = '19-DEC-07'
```

Error at Command Line : 4 Column : 13

Error report -

SQL Error: ORA-00942: table or view does not exist

00942. 00000 - "table or view does not exist"

*Cause:

*Action:

Table JOB_HISTORY1 created.

Activate 'Go to Settings'

The screenshot shows the Oracle SQL Developer interface. The top toolbar includes icons for running queries, saving, and other standard database operations. The 'Query Builder' tab is active, displaying the following SQL statement:

```
INSERT INTO Job_History1
SELECT *
FROM hr.job_history
WHERE end_date = '19-DEC-07';
```

Below the query builder, the 'Script Output' window shows the execution results. It indicates that the task was completed in 0.057 seconds. The output contains an error message:

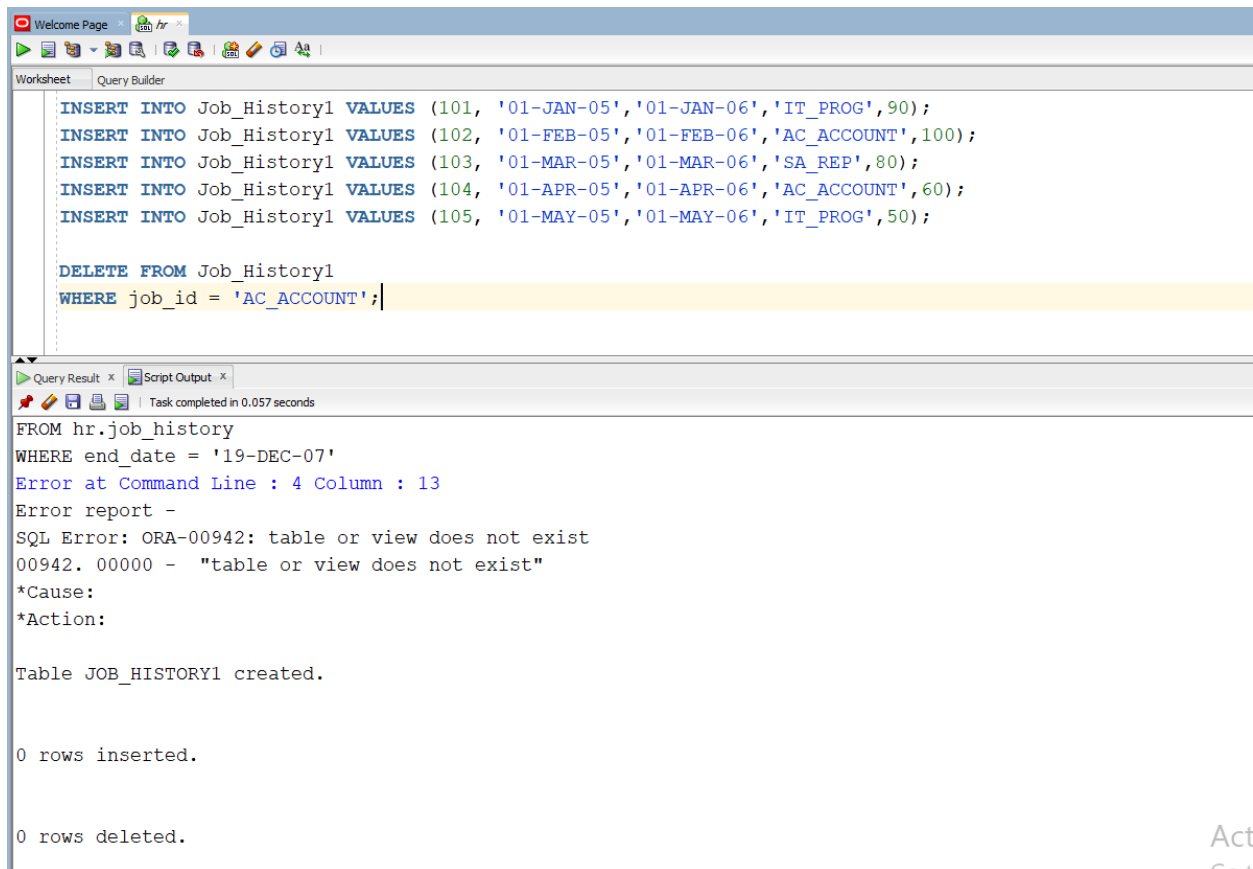
```
00942. 00000 - "table or view does not exist"
*Cause:
*Action:

Error starting at line : 4 in command -
INSERT INTO Job_History1
SELECT *
FROM hr.job_history
WHERE end_date = '19-DEC-07'
Error at Command Line : 4 Column : 13
Error report -
SQL Error: ORA-00942: table or view does not exist
00942. 00000 - "table or view does not exist"
*Cause:
*Action:

Table JOB_HISTORY1 created.

0 rows inserted.
```

TASK 9:



The screenshot displays the Oracle SQL Developer environment. The top pane, titled 'Query Builder', contains the following SQL script:

```
INSERT INTO Job_History1 VALUES (101, '01-JAN-05', '01-JAN-06', 'IT_PROG', 90);
INSERT INTO Job_History1 VALUES (102, '01-FEB-05', '01-FEB-06', 'AC_ACCOUNT', 100);
INSERT INTO Job_History1 VALUES (103, '01-MAR-05', '01-MAR-06', 'SA_REP', 80);
INSERT INTO Job_History1 VALUES (104, '01-APR-05', '01-APR-06', 'AC_ACCOUNT', 60);
INSERT INTO Job_History1 VALUES (105, '01-MAY-05', '01-MAY-06', 'IT_PROG', 50);

DELETE FROM Job_History1
WHERE job_id = 'AC_ACCOUNT';
```

The bottom pane, titled 'Script Output', shows the execution results. It indicates that the task completed in 0.057 seconds. The output includes the following text:

```
FROM hr.job_history
WHERE end_date = '19-DEC-07'
Error at Command Line : 4 Column : 13
Error report -
SQL Error: ORA-00942: table or view does not exist
00942. 00000 - "table or view does not exist"
*Cause:
*Action:

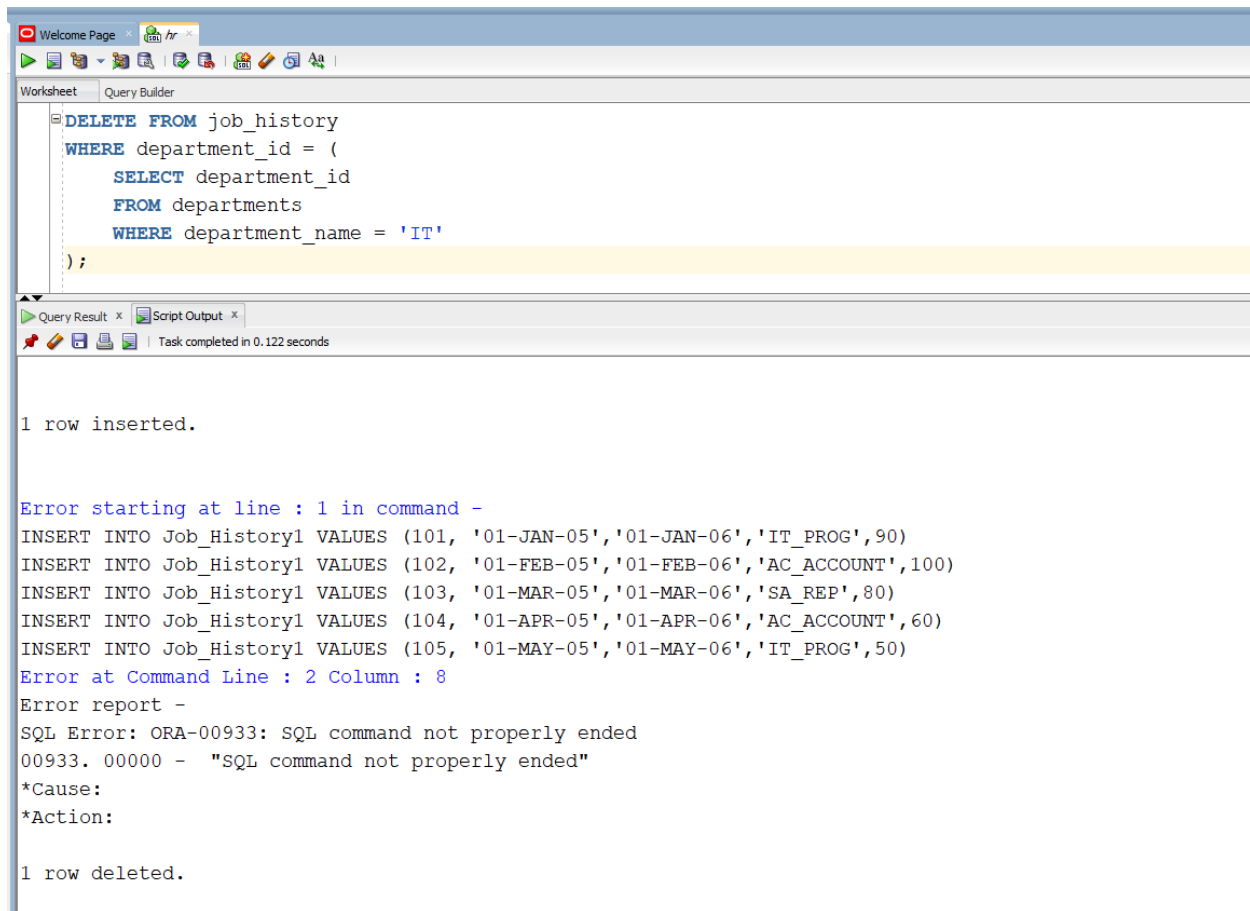
Table JOB_HISTORY1 created.

0 rows inserted.

0 rows deleted.
```

On the right side of the interface, there are two buttons labeled 'Act' and 'Get'.

TASK 10:



The screenshot displays the Oracle SQL Developer environment. The top toolbar includes icons for file operations and execution. The 'Worksheet' tab is active, showing a SQL query in the 'Query Builder' area. The query is a DELETE statement targeting the 'job_history' table, with a subquery in the WHERE clause that selects department IDs from the 'departments' table where the department name is 'IT'. Below the query, the 'Query Result' tab shows the execution output. The first line indicates '1 row inserted.' followed by an error message: 'Error starting at line : 1 in command -'. The error details include five INSERT statements for the 'Job_History1' table, an 'Error at Command Line : 2 Column : 8', and an 'Error report -' section. The error report states: 'SQL Error: ORA-00933: SQL command not properly ended', '00933. 00000 - "SQL command not properly ended"', '*Cause:', and '*Action:'. The final line of the output shows '1 row deleted.'

```
DELETE FROM job_history
WHERE department_id = (
    SELECT department_id
    FROM departments
    WHERE department_name = 'IT'
);
```

1 row inserted.

Error starting at line : 1 in command -

```
INSERT INTO Job_History1 VALUES (101, '01-JAN-05', '01-JAN-06', 'IT_PROG', 90)
INSERT INTO Job_History1 VALUES (102, '01-FEB-05', '01-FEB-06', 'AC_ACCOUNT', 100)
INSERT INTO Job_History1 VALUES (103, '01-MAR-05', '01-MAR-06', 'SA_REP', 80)
INSERT INTO Job_History1 VALUES (104, '01-APR-05', '01-APR-06', 'AC_ACCOUNT', 60)
INSERT INTO Job_History1 VALUES (105, '01-MAY-05', '01-MAY-06', 'IT_PROG', 50)
```

Error at Command Line : 2 Column : 8

Error report -

SQL Error: ORA-00933: SQL command not properly ended

00933. 00000 - "SQL command not properly ended"

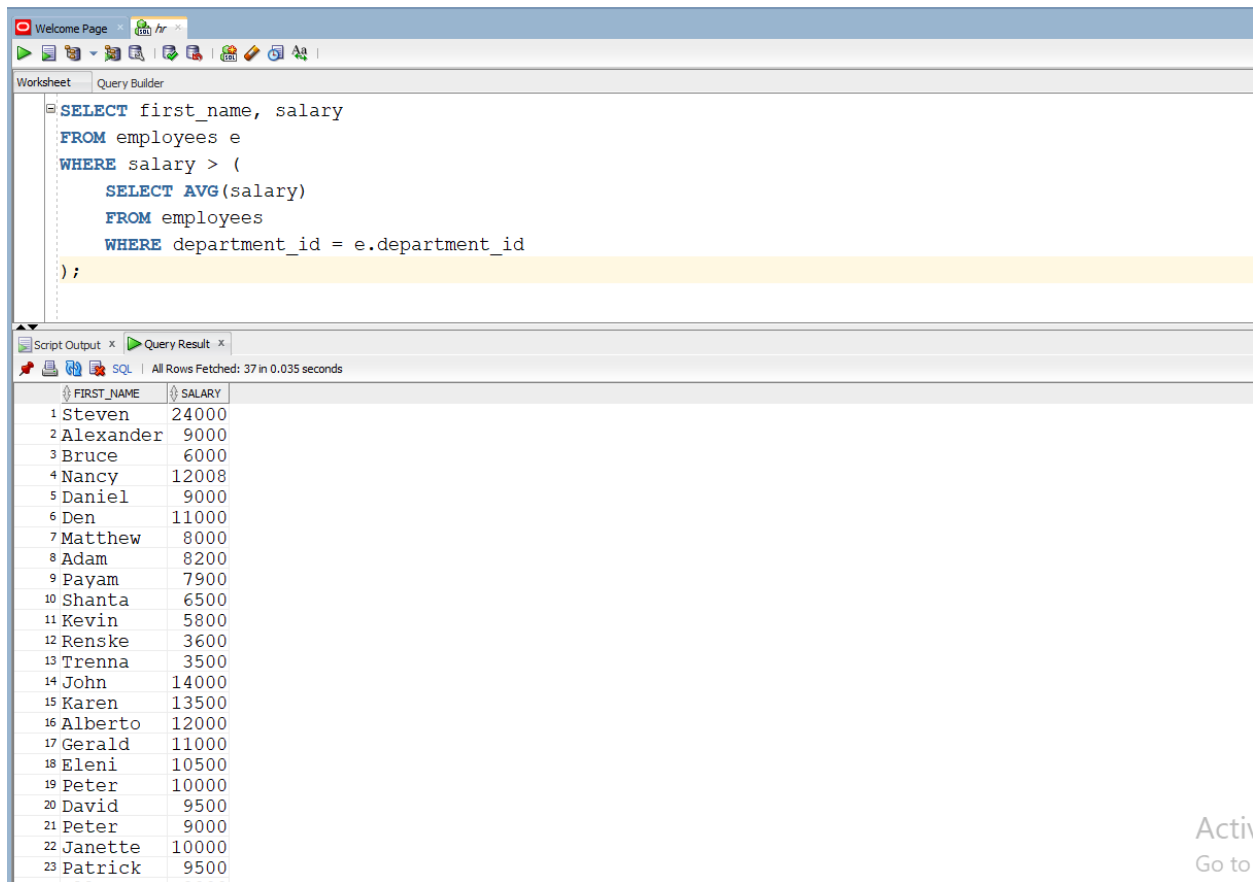
*Cause:

*Action:

1 row deleted.

POST LAB TASK

TASK 1:



The screenshot displays the Oracle SQL Developer environment. The 'Query Builder' tab is active, showing a SQL query that selects the first name and salary of employees whose salary is greater than the average salary of their department. The query is as follows:

```
SELECT first_name, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
);
```

Below the query editor, the 'Query Result' tab shows the output of the query. It indicates that all rows were fetched in 0.035 seconds. The results are displayed in a table with two columns: FIRST_NAME and SALARY.

	FIRST_NAME	SALARY
1	Steven	24000
2	Alexander	9000
3	Bruce	6000
4	Nancy	12008
5	Daniel	9000
6	Den	11000
7	Matthew	8000
8	Adam	8200
9	Payam	7900
10	Shanta	6500
11	Kevin	5800
12	Renske	3600
13	Trenna	3500
14	John	14000
15	Karen	13500
16	Alberto	12000
17	Gerald	11000
18	Eleni	10500
19	Peter	10000
20	David	9500
21	Peter	9000
22	Janette	10000
23	Patrick	9500

On the right side of the interface, there is a partially visible button labeled 'Activ' and 'Go to'.

TASK 2:

The screenshot displays the Oracle SQL Developer environment. The top toolbar includes icons for running queries, saving, and other standard database operations. The 'Worksheet' tab is active, showing a SQL query in the 'Query Builder' pane. The query is as follows:

```
SELECT job_id, MIN(salary) AS min_salary
FROM employees
GROUP BY job_id
HAVING MIN(salary) >= 1000
ORDER BY min_salary ASC;
```

Below the query, the 'Query Result' tab is active, showing the output of the query. The results are displayed in a table with two columns: 'JOB_ID' and 'MIN_SALARY'. The table contains 17 rows of data, sorted by 'MIN_SALARY' in ascending order. The status bar at the bottom indicates 'All Rows Fetched: 17 in 0.071 seconds'.

	JOB_ID	MIN_SALARY
1	ST CLERK	2100
2	PU CLERK	2500
3	SH CLERK	2500
4	IT PROG	4200
5	AD ASST	4400
6	ST MAN	5800
7	SA REP	6100
8	HR REP	6500
9	FI ACCOUNT	6900
10	AC ACCOUNT	8300
11	PR REP	10000
12	SA MAN	10500
13	PU MAN	11000
14	FI MGR	12008
15	AC MGR	12008
16	AD VP	17000
17	AD PRES	24000

TASK 3:

The screenshot shows a database query tool interface. The top bar includes a 'Welcome Page' tab and a toolbar with various icons. Below the toolbar, there are two tabs: 'Worksheet' and 'Query Builder'. The 'Query Builder' tab is active, displaying a SQL query in a text area. The query is as follows:

```
SELECT first_name, department_id
FROM employees
WHERE department_id = (
    SELECT department_id
    FROM employees
    WHERE employee_id = 140
);
```

Below the query editor, there are two tabs: 'Script Output' and 'Query Result'. The 'Query Result' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'FIRST_NAME' and 'DEPARTMENT_ID'. The table contains 23 rows of data, all with a 'DEPARTMENT_ID' of 50.

	FIRST_NAME	DEPARTMENT_ID
1	Matthew	50
2	Adam	50
3	Payam	50
4	Shanta	50
5	Kevin	50
6	Julia	50
7	Irene	50
8	James	50
9	Steven	50
10	Laura	50
11	Mozhe	50
12	James	50
13	TJ	50
14	Jason	50
15	Michael	50
16	Ki	50
17	Hazel	50
18	Renske	50
19	Stephen	50
20	John	50
21	Joshua	50
22	Trenna	50
23	Curtis	50

TASK 4:

The screenshot shows the Oracle SQL Developer interface. The top pane, titled 'Worksheet', contains the following SQL script:

```
CREATE TABLE employees_copy AS
SELECT * FROM employees;
```

The bottom pane, titled 'Script Output', shows the execution results. It indicates that the task was completed in 0.078 seconds. The output includes an error message at the start of the command, followed by five successful INSERT statements into the Job_History1 table, and a final message stating that the Table EMPLOYEES_COPY was created.

```
Error starting at line : 1 in command -
INSERT INTO Job_History1 VALUES (101, '01-JAN-05','01-JAN-06','IT_PROG',90)
INSERT INTO Job_History1 VALUES (102, '01-FEB-05','01-FEB-06','AC_ACCOUNT',100)
INSERT INTO Job_History1 VALUES (103, '01-MAR-05','01-MAR-06','SA_REP',80)
INSERT INTO Job_History1 VALUES (104, '01-APR-05','01-APR-06','AC_ACCOUNT',60)
INSERT INTO Job_History1 VALUES (105, '01-MAY-05','01-MAY-06','IT_PROG',50)
Error at Command Line : 2 Column : 8
Error report -
SQL Error: ORA-00933: SQL command not properly ended
00933. 00000 - "SQL command not properly ended"
*Cause:
*Action:

1 row deleted.

Table EMPLOYEES_COPY created.
```

Welcome Page

Worksheet Query Builder

```
UPDATE employees_copy
SET salary = salary * 1.12
WHERE salary BETWEEN 5000 AND 10000;
```

Script Output x Query Result x

Task completed in 0.123 seconds

Error starting at line : 1 in command -

```
INSERT INTO Job_History1 VALUES (101, '01-JAN-05', '01-JAN-06', 'IT_PROG', 90)
INSERT INTO Job_History1 VALUES (102, '01-FEB-05', '01-FEB-06', 'AC_ACCOUNT', 100)
INSERT INTO Job_History1 VALUES (103, '01-MAR-05', '01-MAR-06', 'SA_REP', 80)
INSERT INTO Job_History1 VALUES (104, '01-APR-05', '01-APR-06', 'AC_ACCOUNT', 60)
INSERT INTO Job_History1 VALUES (105, '01-MAY-05', '01-MAY-06', 'IT_PROG', 50)
```

Error at Command Line : 2 Column : 8

Error report -

SQL Error: ORA-00933: SQL command not properly ended

00933. 00000 - "SQL command not properly ended"

*Cause:

*Action:

1 row deleted.

Table EMPLOYEES_COPY created.

42 rows updated.

TASK 5:

The screenshot shows the SQL Developer interface with a query in the Query Builder. The query is designed to select the top 10% of employees based on their salary in descending order.

```
SELECT *  
FROM (  
    SELECT first_name, salary  
    FROM employees  
    ORDER BY salary DESC  
)  
WHERE ROWNUM <= (  
    SELECT COUNT(*)*0.1 FROM employees  
) ;
```

The Query Result pane displays the following data:

	FIRST_NAME	SALARY
1	Steven	24000
2	Neena	17000
3	Lex	17000
4	John	14000
5	Karen	13500
6	Nancy	12008
7	Shelley	12008
8	Alberto	12000
9	Lisa	11500
10	Den	11000

TASK 6:

The screenshot shows the SQL Developer interface with a query in the Query Builder. The query is designed to find departments that have more than one location.

```
SELECT location_id, COUNT(department_id)  
FROM departments  
GROUP BY location_id  
HAVING COUNT(department_id) > 1 ;
```

The Query Result pane displays the following data:

	LOCATION_ID	COUNT(DEPARTMENT_ID)
1	1700	21

TASK 7:

The screenshot shows a SQL query editor window with a 'Query Builder' tab. The query is as follows:

```
SELECT job_id, MIN(salary) AS min_salary
FROM employees
WHERE job_id IS NOT NULL
GROUP BY job_id
HAVING MIN(salary) != 1500
ORDER BY min_salary DESC;
```

Below the query editor, the 'Query Result' tab is active, displaying 17 rows of data. The status bar indicates 'All Rows Fetched: 17 in 0.008 seconds'.

	JOB_ID	MIN_SALARY
1	AD PRES	24000
2	AD VP	17000
3	AC MGR	12008
4	FI MGR	12008
5	PU MAN	11000
6	SA MAN	10500
7	PR REP	10000
8	AC ACCOUNT	8300
9	FI ACCOUNT	6900
10	HR REP	6500
11	SA REP	6100
12	ST MAN	5800
13	AD ASST	4400
14	IT PROG	4200
15	SH CLERK	2500
16	PU CLERK	2500
17	ST CLERK	2100

TASK 8:

The screenshot shows the SQL Developer interface. The 'Query Builder' tab is active, displaying the following SQL query:

```
SELECT first name, department_id
FROM employees
WHERE department_id = (
    SELECT department_id
    FROM employees
    WHERE employee_id = 130
);
```

The 'Query Result' tab shows the results of the query. It indicates that all rows were fetched in 0.004 seconds. The results are displayed in a table with two columns: FIRST_NAME and DEPARTMENT_ID.

	FIRST_NAME	DEPARTMENT_ID
1	Matthew	50
2	Adam	50
3	Payam	50
4	Shanta	50
5	Kevin	50
6	Julia	50
7	Irene	50
8	James	50
9	Steven	50
10	Laura	50
11	Mozhe	50
12	James	50
13	TJ	50
14	Jason	50
15	Michael	50
16	Ki	50
17	Hazel	50
18	Renske	50
19	Stephen	50
20	John	50
21	Joshua	50
22	Trenna	50
23	Curtis	50
24	Randall	50
25	Peter	50

TASK 9:

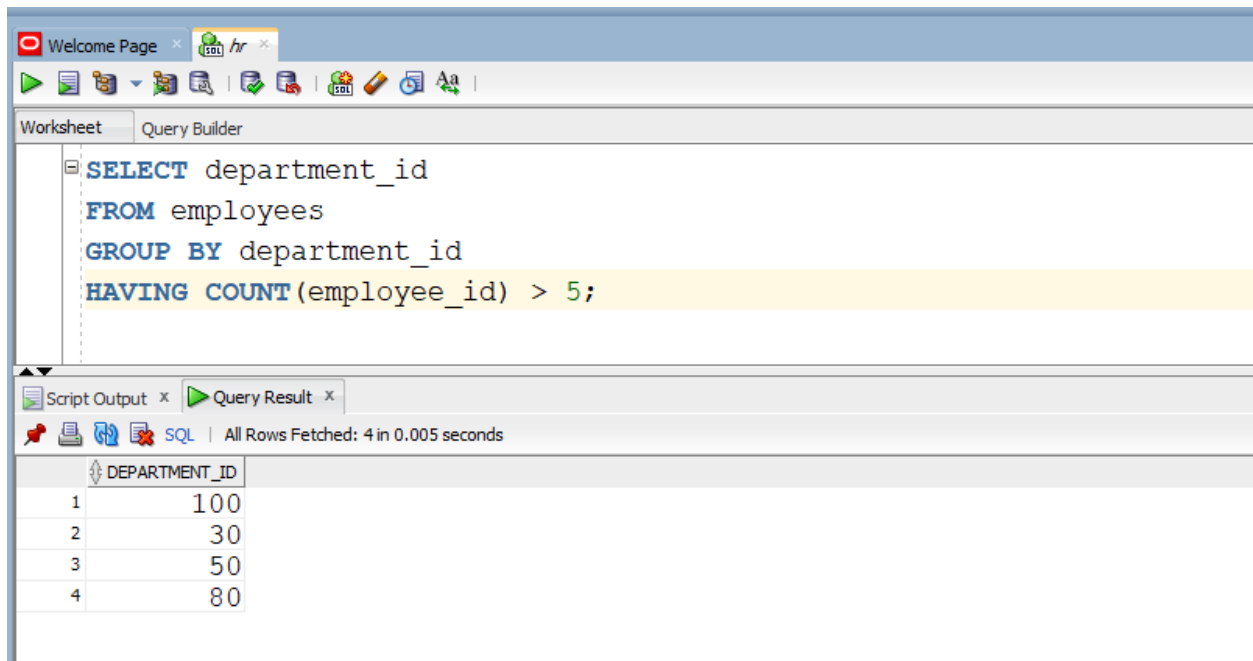
The screenshot shows the SQL Developer interface. The 'Query Builder' tab is active, displaying the following SQL query:

```
SELECT *
FROM employees
WHERE department_id != 30
AND salary > ALL (
    SELECT salary
    FROM employees
    WHERE department_id = 30
);
```

The 'Query Result' tab shows the results of the query. It indicates that all rows were fetched in 0.074 seconds. The results are displayed in a table with 11 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, JOB_ID, SALARY, COMMISSION_PCT, MANAGER_ID, and DEPARTMENT_ID.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	168	Lisa	Ozer	LOZER	011.44.1343.929268	11-MAR-05	SA REP	11500	0.25	148	80
2	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10-MAR-05	SA MAN	12000	0.3	100	80
3	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-02	AC MGR	12008	(null)	101	110
4	108	Nancy	Greenberg	NGREENBE	515.124.4569	17-AUG-02	FI MGR	12008	(null)	101	100
5	146	Karen	Partners	KPARTNER	011.44.1344.467268	05-JAN-05	SA MAN	13500	0.3	100	80
6	145	John	Russell	JRUSSEL	011.44.1344.429268	01-OCT-04	SA MAN	14000	0.4	100	80
7	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-01	AD VP	17000	(null)	100	90
8	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-05	AD VP	17000	(null)	100	90
9	100	Steven	King	SKING	515.123.4567	17-JUN-03	AD PRES	24000	(null)	(null)	90

TASK 10:



The screenshot shows an SQL IDE interface. The top toolbar includes icons for running queries, saving, and other standard database operations. The main workspace is divided into two panes: 'Worksheet' and 'Query Builder'. The 'Worksheet' pane contains the following SQL query:

```
SELECT department_id
FROM employees
GROUP BY department_id
HAVING COUNT(employee_id) > 5;
```

The 'Query Result' pane shows the output of the query. It displays a table with one column, 'DEPARTMENT_ID', and four rows of data. The status bar indicates that all rows were fetched successfully in 0.005 seconds.

	DEPARTMENT_ID
1	100
2	30
3	50
4	80